

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ  
ФЕДЕРАЦИИ  
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬ-  
НОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ  
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ ТЕХНОЛОГИЧЕСКИЙ  
УНИВЕРСИТЕТ «МИСИС»

*ИНСТИТУТ                      Информационных компьютерных наук (ИКН)*  
*КАФЕДРА    Институт компьютерных наук (ИКН)*  
*НАПРАВЛЕНИЕ 09.03.04    ГРУППА БИВТ-24-7*

## КУРСОВОЙ ПРОЕКТ

*ПО КУРСУ: Клиент серверные приложения .*

*ТЕМА: Разработка информационной системы управления магазином детских  
игрушек «Toy Shop»*

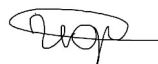
*Студент Иордан Дмитрий Сергеевич*

*Проверил: ктн, снс, доцент Микитенко Игорь Иванович*

*Шелудяков Петр Алексеевич*

*Абросимов Никита Андреевич*

*Подпись:*



*Оценка выполнения курсового проекта \_\_\_\_\_*

## СОДЕРЖАНИЕ

|                                                 |       |
|-------------------------------------------------|-------|
| 1. Предметная область.....                      | 3-4   |
| 2. Постановка задачи.....                       | 4-6   |
| 3. Описание архитектуры системы.....            | 6-9   |
| 4. Описание структуры базы данных.....          | 9-14  |
| 5. Описание серверной части.....                | 14-18 |
| 6. Описание клиентской части.....               | 18-24 |
| 7. Развертывание и проверка работы системы..... | 24-26 |
| 8. Заключение.....                              | 27-28 |
| 9. Список использованных источников.....        | 29    |
| 10. Приложение.....                             | 30-31 |

# **1. ПРЕДМЕТНАЯ ОБЛАСТЬ**

Предметной областью данной курсовой работы является деятельность небольшого магазина детских игрушек. В таком магазине требуется хранить информацию о товарах, категориях, производителях, покупателях и заказах. Также важно контролировать остатки на складе, потому что покупатель не должен оформить заказ на товар, которого уже нет в наличии.

При ручном ведении учета могут возникать разные проблемы. Например, продавец может забыть обновить количество товара на складе, потерять информацию о покупателе или не сразу увидеть, какой заказ уже доставлен, а какой только создан. Из-за этого работа магазина становится менее удобной и занимает больше времени.

Для решения этих задач была разработана веб-система «Toy Shop». Она позволяет пользователю просматривать каталог игрушек и оформлять заказы, а администратору — управлять товарами, просматривать заказы, менять их статусы, получать статистику и смотреть журнал действий пользователей.

## **1.1 Основные участники системы**

В системе предусмотрены два типа пользователей: обычный пользователь и администратор. Обычный пользователь работает с каталогом и может оформить заказ. Администратор имеет расширенные права и может управлять товарами, заказами и служебной информацией.

- Пользователь — просматривает каталог игрушек, ищет нужный товар и оформляет заказ.
- Администратор — добавляет, изменяет и удаляет товары, просматривает все заказы, меняет статусы заказов, смотрит статистику и логи.
- Покупатель — человек, данные которого сохраняются при оформлении заказа: ФИО, email и телефон.

## 1.2 Основные сущности предметной области

В рамках предметной области можно выделить несколько сущностей. Они используются как в базе данных, так и в логике приложения.

- Игрушка — основной товар магазина, который имеет название, описание, цену, возрастное ограничение и остаток на складе.
- Категория — группа товаров, например конструкторы, куклы, машинки, настольные игры и мягкие игрушки.
- Производитель — организация или бренд, который выпускает игрушки.
- Покупатель — клиент магазина, который оформляет заказ.
- Заказ — запись о покупке конкретной игрушки определенным покупателем.
- Пользователь системы — учетная запись для входа в приложение.
- Журнал действий — записи о важных действиях пользователей, например вход в систему или изменение товара.

## 1.3 Статусы заказов

Для заказов в системе используются статусы. Они нужны, чтобы администратор мог понимать, на каком этапе находится заказ. В проекте используются следующие статусы:

- new — новый заказ;
- confirmed — заказ подтвержден;
- shipped — заказ отправлен;
- delivered — заказ доставлен;
- cancelled — заказ отменен.

Использование статусов делает работу с заказами более понятной. Например, администратор может быстро увидеть новые заказы и поменять их состояние после обработки.

## 2. ПОСТАНОВКА ЗАДАЧИ

Целью курсовой работы является разработка веб-ориентированной информационной системы управления магазином детских игрушек. Система должна помогать хранить каталог товаров, оформлять заказы и выполнять административные функции.

### 2.1 Задачи проекта

Для достижения поставленной цели необходимо решить следующие задачи:

- Спроектировать реляционную базу данных PostgreSQL для хранения данных магазина.
- Создать таблицы для пользователей, категорий, производителей, покупателей, игрушек, заказов и журнала действий.
- Реализовать представления, функции и процедуры на стороне базы данных.
- Разработать серверную часть на языке Python с использованием FastAPI.
- Реализовать авторизацию пользователей с использованием JWT-токенов.
- Создать REST API для работы с каталогом, заказами, покупателями, статистикой и логами.
- Разработать клиентскую часть на React, которая будет обращаться к серверу через HTTP-запросы.
- Добавить разделение прав доступа для администратора и обычного пользователя.
- Выполнить контейнеризацию проекта с использованием Docker и Docker Compose.
- Проверить работу основных сценариев: вход, просмотр каталога, поиск, оформление заказа и управление товарами.

## 2.2 Основные требования к системе

К системе предъявляются следующие требования:

- наличие удобного веб-интерфейса;
- возможность входа по логину и паролю;
- разделение функций администратора и пользователя;
- просмотр каталога игрушек;
- поиск товара по названию и описанию;
- оформление заказа с проверкой количества товара на складе;
- ведение журнала действий пользователей;
- получение статистики по заказам;
- возможность запуска проекта через Docker Compose.

## 2.3 Ожидаемый результат

В результате выполнения работы должно получиться приложение, которое можно запустить локально и использовать через браузер. После запуска пользователь должен иметь возможность войти в систему, перейти в каталог, выбрать товар и оформить заказ. Администратор должен иметь возможность управлять товарами и заказами.

# 3. ОПИСАНИЕ АРХИТЕКТУРЫ СИСТЕМЫ

## 3.1 Общая архитектура

Разрабатываемая система построена по трехуровневой архитектуре «клиент — сервер — база данных». Такой подход часто используется в веб-приложениях, потому что позволяет разделить внешний интерфейс, обработку запросов и хранение информации.

Схема взаимодействия компонентов имеет следующий вид:

**Браузер → React → FastAPI → PostgreSQL**

## Архитектура системы Toy Shop

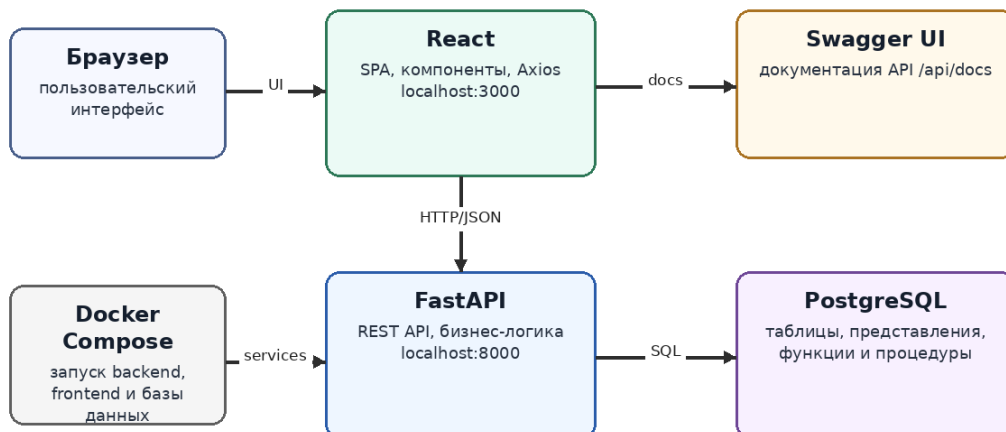


Рисунок 1 — Архитектура системы Toy Shop

Пользователь взаимодействует только с клиентской частью. React-приложение показывает страницы, кнопки, формы и таблицы. Когда пользователь выполняет действие, например нажимает кнопку поиска или отправляет форму заказа, клиентская часть формирует HTTP-запрос к серверу.

Серверная часть на FastAPI принимает запрос, проверяет данные, выполняет бизнес-логику и обращается к базе данных. База данных PostgreSQL хранит всю информацию и возвращает результат серверу. После этого сервер отправляет ответ клиентской части в формате JSON.

### 3.2 Уровень представления

Уровень представления реализован на React. Он отвечает за отображение страниц приложения, работу с формами, модальными окнами и таблицами. В интерфейсе есть экран входа, главная страница, каталог товаров, раздел заказов, статистика и журнал действий.

### 3.3 Уровень бизнес-логики

Уровень бизнес-логики реализован с помощью FastAPI. В этой части приложения описаны маршруты API, схемы данных, авторизация, проверка прав администратора и работа с базой данных. Например, при создании заказа сервер проверяет, существует ли игрушка и хватает ли ее на складе.

### 3.4 Уровень данных

Уровень данных представлен базой PostgreSQL. В ней находятся таблицы, представления, хранимые функции и процедуры. Такая структура позволяет не только хранить данные, но и выполнять часть операций на стороне базы данных, например получать статистику заказов или изменять статус заказа через процедуру.

### 3.5 Технологический стек

Таблица 1 — Технологический стек проекта

| Компонент          | Используемая технология     | Назначение                                          |
|--------------------|-----------------------------|-----------------------------------------------------|
| Клиентская часть   | React, JavaScript, Axios    | Создание интерфейса и отправка HTTP-запросов        |
| Серверная часть    | Python, FastAPI, SQLAlchemy | Обработка запросов и работа с базой данных          |
| База данных        | PostgreSQL                  | Хранение данных, представления, функции и процедуры |
| Авторизация        | JWT, passlib, bcrypt        | Вход пользователей и защита API                     |
| Контейнеризация    | Docker, Docker Compose      | Запуск всех частей проекта одной командой           |
| Веб-сервер backend | Uvicorn                     | Запуск FastAPI-приложения                           |

### 3.6 Взаимодействие компонентов

Общий порядок работы приложения можно описать следующим образом:

- Пользователь открывает веб-приложение в браузере.
- React показывает экран входа и отправляет логин и пароль на сервер.
- FastAPI проверяет пользователя в базе данных и создает JWT-токен.



- Клиент сохраняет токен и добавляет его в заголовок Authorization при последующих запросах.
- Пользователь выбирает нужный раздел приложения.
- React отправляет запрос к API, например для получения игрушек или заказов.
- FastAPI выполняет SQL-запрос или вызывает представление, функцию или процедуру в PostgreSQL.
- База данных возвращает результат серверу.
- Сервер отправляет данные клиенту в формате JSON.
- React обновляет интерфейс без перезагрузки страницы.

## **4. ОПИСАНИЕ СТРУКТУРЫ БАЗЫ ДАННЫХ**

### **4.1 Логическая модель**

База данных построена на основе реляционной модели. Каждая таблица отвечает за отдельную сущность предметной области. Между таблицами используются связи через внешние ключи.

## Логическая модель базы данных

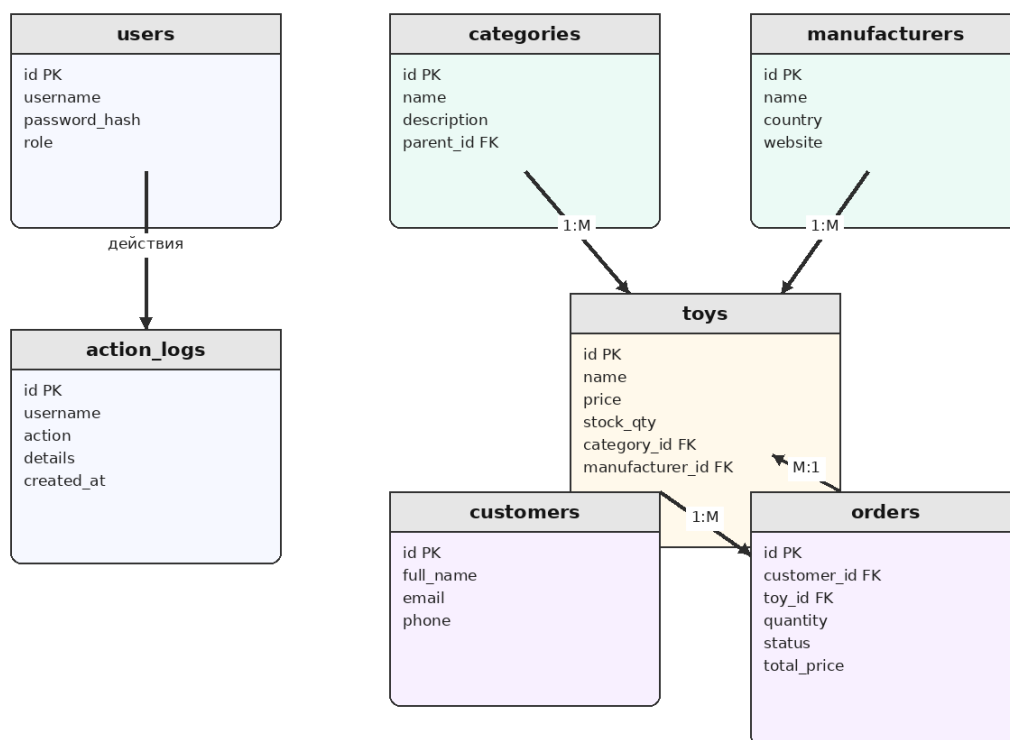


Рисунок 2 — Логическая модель базы данных

Главной таблицей для каталога является таблица `toys`. Она связана с таблицами `categories` и `manufacturers`. Таблица `orders` связана с таблицами `customers` и `toys`, потому что каждый заказ относится к конкретному покупателю и конкретной игрушке. Таблица `action_logs` используется для хранения важных действий пользователей.

## 4.2 Таблицы базы данных

### Таблица `users`

Таблица `users` предназначена для хранения учетных записей пользователей системы. В ней сохраняются логин, хеш пароля, роль пользователя и дата создания учетной записи. Роль используется для разделения прав доступа.

Таблица 2 — Структура `users`

| Поле            | Описание                              |
|-----------------|---------------------------------------|
| <code>id</code> | Уникальный идентификатор пользователя |

|               |                                   |
|---------------|-----------------------------------|
| username      | Логин пользователя                |
| password_hash | Хеш пароля                        |
| role          | Роль пользователя: admin или user |
| created_at    | Дата создания записи              |

### Таблица categories

Таблица categories хранит категории игрушек. Категории нужны для группировки товаров и более удобной навигации по каталогу.

Таблица 3 — Структура categories

| Поле        | Описание                           |
|-------------|------------------------------------|
| id          | Уникальный идентификатор категории |
| name        | Название категории                 |
| description | Описание категории                 |
| parent_id   | Ссылка на родительскую категорию   |

### Таблица manufacturers

Таблица manufacturers содержит информацию о производителях игрушек. В карточке товара отображается название производителя и страна.

Таблица 4 — Структура manufacturers

| Поле    | Описание                               |
|---------|----------------------------------------|
| id      | Уникальный идентификатор производителя |
| name    | Название производителя                 |
| country | Страна производителя                   |
| website | Сайт производителя                     |

### Таблица customers

Таблица customers предназначена для хранения покупателей. При оформлении заказа пользователь вводит ФИО, email и телефон. Email сделан уникальным, чтобы не создавать одинаковых покупателей повторно.

Таблица 5 — Структура customers

| Поле      | Описание                            |
|-----------|-------------------------------------|
| id        | Уникальный идентификатор покупателя |
| full_name | ФИО покупателя                      |
| email     | Электронная почта                   |
| phone     | Телефон                             |

|            |                      |
|------------|----------------------|
| address    | Адрес покупателя     |
| created_at | Дата создания записи |

## Таблица toys

Таблица toys является основной таблицей каталога. В ней хранится информация о каждой игрушке: название, описание, цена, возраст от которого подходит товар, количество на складе, категория и производитель.

*Таблица 6 — Структура toys*

| Поле            | Описание                         |
|-----------------|----------------------------------|
| id              | Уникальный идентификатор игрушки |
| name            | Название игрушки                 |
| description     | Описание товара                  |
| price           | Цена                             |
| age_from        | Минимальный возраст              |
| stock_qty       | Остаток на складе                |
| category_id     | Ссылка на категорию              |
| manufacturer_id | Ссылка на производителя          |
| created_at      | Дата добавления товара           |

## Таблица orders

Таблица orders хранит информацию о заказах. Каждый заказ связан с покупателем и игрушкой. Также в заказе указывается количество, статус, итоговая сумма и даты создания и обновления.

*Таблица 7 — Структура orders*

| Поле        | Описание                        |
|-------------|---------------------------------|
| id          | Уникальный идентификатор заказа |
| customer_id | Ссылка на покупателя            |
| toy_id      | Ссылка на игрушку               |
| quantity    | Количество товара               |
| status      | Статус заказа                   |
| total_price | Итоговая сумма заказа           |
| created_at  | Дата создания                   |
| updated_at  | Дата последнего изменения       |

## Таблица action\_logs

Таблица action\_logs используется для журнала действий. Она нужна, чтобы администратор мог видеть важные события, например вход пользователя, добавление товара, изменение заказа или очистку логов.

Таблица 8 — Структура action\_logs

| Поле       | Описание                        |
|------------|---------------------------------|
| id         | Уникальный идентификатор записи |
| username   | Имя пользователя                |
| action     | Название действия               |
| details    | Дополнительная информация       |
| created_at | Дата и время действия           |

## 4.3 Представления

В базе данных используются представления. Они позволяют заранее объединить данные из нескольких таблиц и упростить запросы на серверной части.

Таблица 9 — Представления базы данных

| Представление  | Назначение                                                                                       |
|----------------|--------------------------------------------------------------------------------------------------|
| v_toys_full    | Возвращает игрушки вместе с названием категории, названием производителя и страной производителя |
| v_order_stats  | Показывает количество заказов и выручку по каждому статусу                                       |
| v_popular_toys | Показывает популярные игрушки по количеству заказов                                              |

## 4.4 Хранимые функции

Функции в PostgreSQL используются для получения данных и вычислений. В проекте реализованы следующие функции:

Таблица 10 — Хранимые функции

| Функция                     | Назначение                                     |
|-----------------------------|------------------------------------------------|
| fn_toys_by_category(cat_id) | Возвращает список игрушек выбранной категории  |
| fn_search_toys(query)       | Выполняет поиск игрушек по названию и описанию |

|                                               |                                                     |
|-----------------------------------------------|-----------------------------------------------------|
| <code>fn_customer_order_count(cust_id)</code> | Возвращает количество заказов указанного покупателя |
|-----------------------------------------------|-----------------------------------------------------|

## 4.5 Хранимые процедуры

Хранимые процедуры используются для изменения данных. Они удобны для операций, которые должны выполняться на уровне базы данных.

*Таблица 11 — Хранимые процедуры*

| Процедура                                                 | Назначение                                                    |
|-----------------------------------------------------------|---------------------------------------------------------------|
| <code>sp_change_order_status(order_id, new_status)</code> | Изменяет статус заказа и дату обновления                      |
| <code>sp_restock_toys(toy_id, add_qty)</code>             | Увеличивает остаток товара на складе                          |
| <code>sp_cancel_old_orders(days_old)</code>               | Отменяет старые новые заказы, которые долго не обрабатывались |

## 4.6 Начальные данные

Для демонстрации работы системы в базе данных предусмотрены начальные данные. В проекте добавлены две учетные записи: `admin` с ролью администратора и `user1` с ролью обычного пользователя. Также добавлены категории игрушек, производители, покупатели, игрушки и несколько тестовых заказов.

Наличие тестовых данных удобно для проверки приложения сразу после запуска. Например, можно войти под администратором, увидеть список товаров, проверить статистику и изменить статус заказа.

## 5. ОПИСАНИЕ СЕРВЕРНОЙ ЧАСТИ

Серверная часть разработана на языке Python с использованием фреймворка FastAPI. FastAPI удобен тем, что позволяет быстро создавать REST API и автоматически формирует документацию Swagger UI. Для подключения к базе данных используется SQLAlchemy.

## 5.1 Основные зависимости

В файле requirements.txt указаны зависимости backend-части. Среди них FastAPI, Uvicorn, SQLAlchemy, драйвер psycopg2-binary для PostgreSQL, python-jose для JWT-токенов, passlib и bcrypt для работы с паролями.

*Таблица 12 — Основные зависимости серверной части*

| Библиотека      | Назначение                      |
|-----------------|---------------------------------|
| fastapi         | Создание API                    |
| uvicorn         | Запуск веб-сервера              |
| sqlalchemy      | Работа с базой данных           |
| psycopg2-binary | Подключение к PostgreSQL        |
| python-jose     | Создание и проверка JWT-токенов |
| passlib, bcrypt | Проверка хешей паролей          |

## 5.2 Авторизация

В приложении реализована авторизация по логину и паролю. Пользователь отправляет данные на маршрут /api/auth/login. Сервер ищет пользователя в таблице users и проверяет пароль по хешу. Если данные верные, сервер создает JWT-токен.

Токен содержит имя пользователя, роль и время окончания действия. В проекте срок действия токена составляет 24 часа. После входа клиентская часть добавляет токен в заголовок Authorization при обращении к защищенным методам API.

Для ограничения доступа используется функция require\_admin. Если пользователь не является администратором, сервер возвращает ошибку доступа. Так защищены методы управления товарами, заказами, статистикой и логами.

## 5.3 Работа с товарами

Для работы с товарами реализованы методы получения списка игрушек, поиска, получения игрушек по категории, просмотра одного товара, создания, изменения и удаления товара. Обычный пользователь может

просматривать товары, а администратор дополнительно может добавлять, редактировать и удалять их.

*Таблица 13 — API для работы с игрушками*

| Метод  | Адрес                         | Назначение                             |
|--------|-------------------------------|----------------------------------------|
| GET    | /api/toys                     | Получить список игрушек                |
| GET    | /api/toys/search/{query_text} | Найти игрушки по названию или описанию |
| GET    | /api/toys/category/{cat_id}   | Получить игрушки выбранной категории   |
| GET    | /api/toys/{toy_id}            | Получить одну игрушку                  |
| POST   | /api/toys                     | Создать игрушку, только администратор  |
| PUT    | /api/toys/{toy_id}            | Изменить игрушку, только администратор |
| DELETE | /api/toys/{toy_id}            | Удалить игрушку, только администратор  |

## 5.4 Работа с покупателями

При оформлении заказа клиентская часть сначала создает или получает покупателя. В методе создания покупателя используется проверка по email. Если покупатель с таким email уже существует, система возвращает его id, а не создает дубликат.

## 5.5 Работа с заказами

Заказы являются важной частью системы. При создании заказа сервер сначала проверяет, существует ли выбранная игрушка. Затем проверяется остаток на складе. Если пользователь пытается заказать больше, чем есть в наличии, сервер возвращает ошибку «Недостаточно товара на складе».

Если товара достаточно, сервер рассчитывает итоговую сумму заказа, создает запись в таблице orders и уменьшает количество товара на складе. Это позволяет поддерживать актуальные остатки после оформления заказов.

*Таблица 14 — API для работы с заказами*

| Метод | Адрес       | Назначение                                    |
|-------|-------------|-----------------------------------------------|
| GET   | /api/orders | Получить список заказов, только администратор |



|      |                               |                                              |
|------|-------------------------------|----------------------------------------------|
| POST | /api/orders                   | Создать заказ                                |
| POST | /api/orders/{order_id}/status | Изменить статус заказа, только администратор |

## 5.6 Справочники и статистика

Серверная часть также предоставляет методы для получения справочников категорий и производителей. Эти данные используются на клиентской стороне, например при добавлении или редактировании игрушки.

Для администратора реализована статистика. Метод /api/stats/orders возвращает количество заказов и выручку по статусам. Метод /api/stats/popular возвращает топ популярных игрушек. Такие данные могут использоваться для анализа продаж.

*Таблица 15 — API справочников и статистики*

| Метод | Адрес                                | Назначение                             |
|-------|--------------------------------------|----------------------------------------|
| GET   | /api/categories                      | Получить категории                     |
| GET   | /api/manufacturers                   | Получить производителей                |
| GET   | /api/stats/orders                    | Получить статистику заказов            |
| GET   | /api/stats/popular                   | Получить популярные игрушки            |
| GET   | /api/stats/customer/{cust_id}/orders | Получить количество заказов покупателя |

## 5.7 Журнал действий

Журнал действий нужен для контроля работы системы. В него записываются такие действия, как вход пользователя, создание товара, изменение товара, удаление товара, создание заказа, изменение статуса заказа и очистка логов. Просмотр и очистка логов доступны только администратору.

*Таблица 16 — API журнала действий*

| Метод  | Адрес     | Назначение                        |
|--------|-----------|-----------------------------------|
| GET    | /api/logs | Получить последние записи журнала |
| DELETE | /api/logs | Очистить журнал действий          |

## 5.8 Документация API

FastAPI автоматически формирует документацию API. После запуска backend документацию можно открыть в браузере. Это удобно для проверки методов без отдельной программы, потому что Swagger UI позволяет отправлять запросы прямо из браузера.

## 6. ОПИСАНИЕ КЛИЕНТСКОЙ ЧАСТИ

Клиентская часть разработана на React. Она является одностраничным приложением и работает через компонент App. Для отправки запросов на сервер используется библиотека Axios.

### 6.1 Экран входа

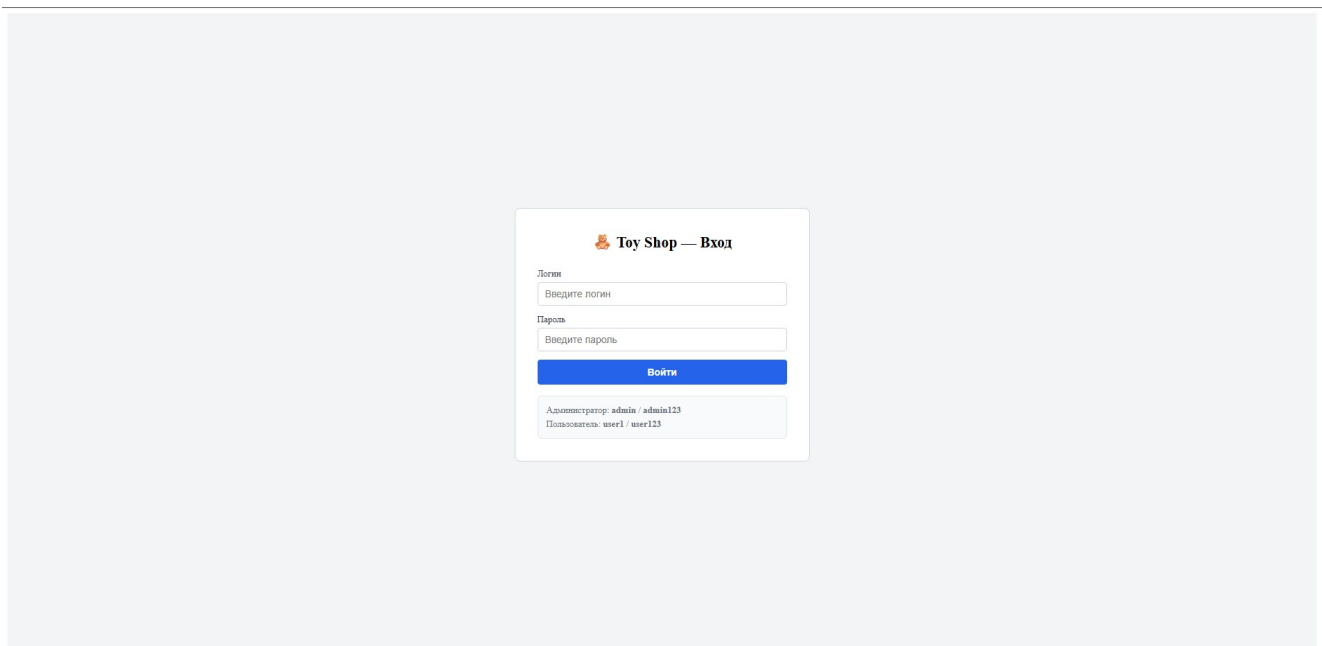


Рисунок 3 — Экран входа

При первом открытии приложения пользователь видит экран входа. На нем есть поля для логина и пароля, кнопка входа и подсказки с тестовыми учетными записями. Если пользователь не ввел логин или пароль, интерфейс показывает ошибку. Если сервер возвращает ошибку авторизации, она также отображается пользователю.

## 6.2 Главная страница

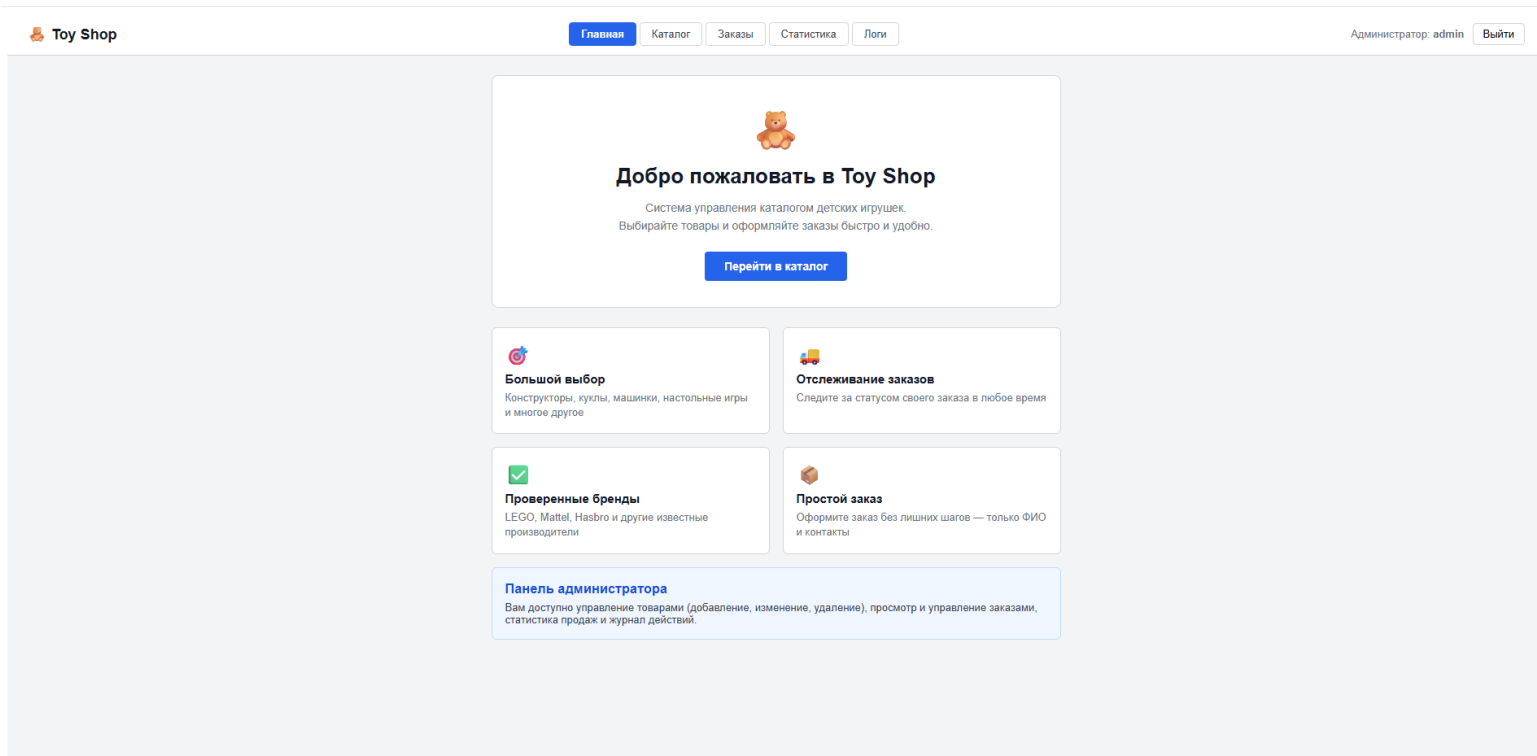


Рисунок 4 — Главная страница

После успешного входа пользователь попадает на главную страницу. На ней отображается приветствие и краткое описание системы. Для перехода к товарам есть кнопка «Перейти в каталог». Также на главной странице показаны преимущества: большой выбор, отслеживание заказов, проверенные бренды и простой заказ.

## 6.3 Каталог товаров

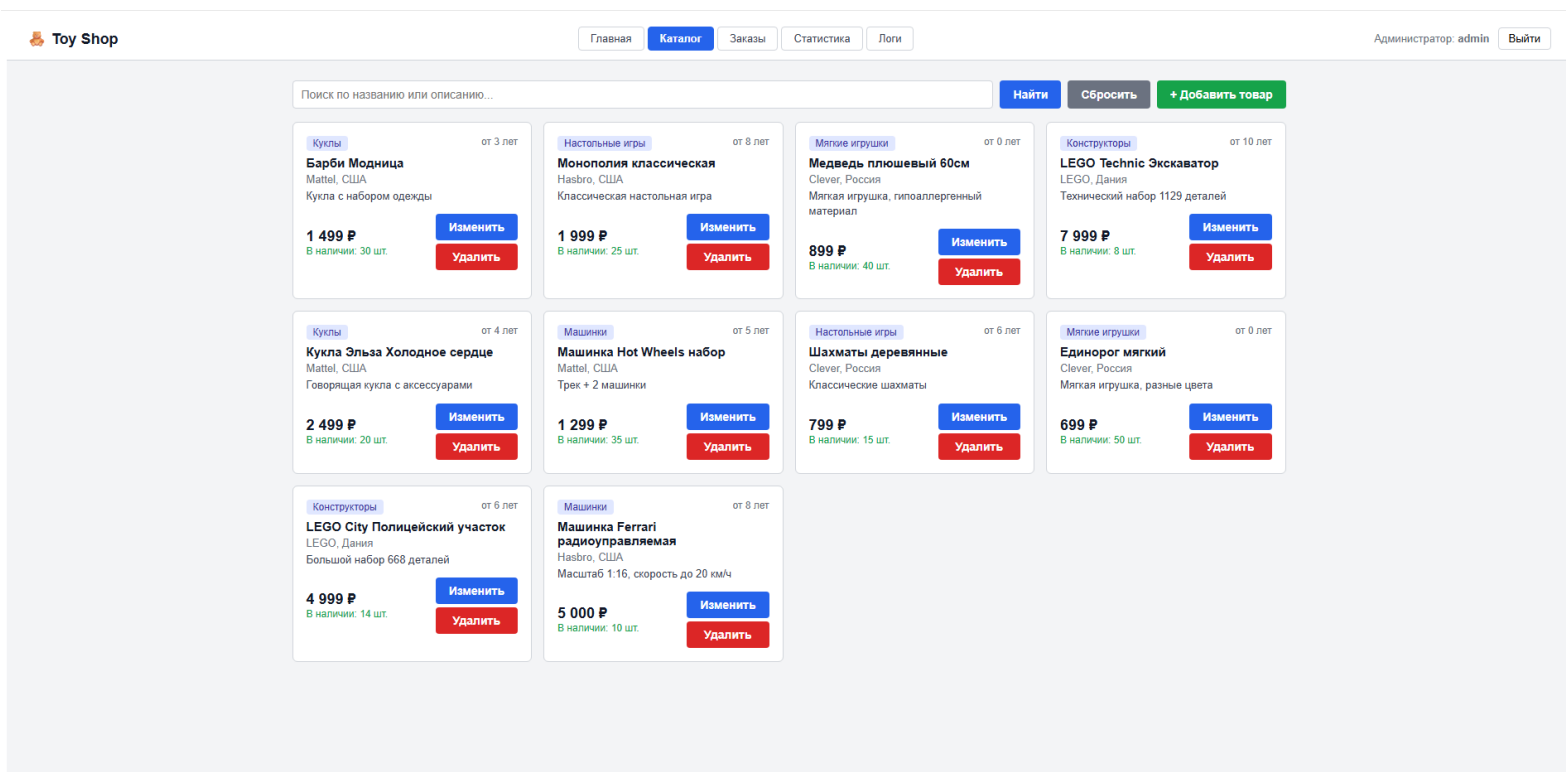


Рисунок 5 — Каталог товаров

Каталог товаров отображает игрушки в виде карточек. В каждой карточке показываются категория, возраст, название, производитель, описание, цена и количество на складе. Если товар есть в наличии, обычный пользователь может нажать кнопку «Заказать». Если товара нет, кнопка становится недоступной.

Для администратора в карточке отображаются кнопки «Изменить» и «Удалить». Также администратору доступна кнопка добавления нового товара. Форма добавления и редактирования открывается в модальном окне.

## 6.4 Поиск

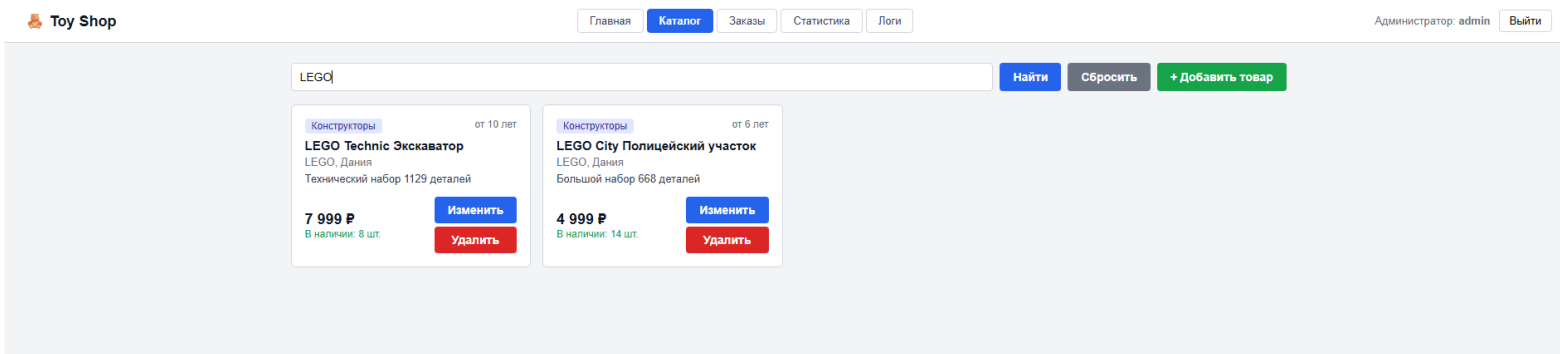


Рисунок 6 — Поиск

В каталоге есть поле поиска. Пользователь может ввести часть названия или описания товара и нажать кнопку «Найти». После этого React отправляет запрос на сервер, а сервер возвращает подходящие товары. Также есть кнопка «Сбросить», которая снова загружает полный список игрушек.

## 6.5 Оформление заказа

**Барби Модница**  
**1 499 Р за шт.**×

ФИО \*

Иордан Дмитрий Сергеевич

Email \*

m2412976@edu.misis.ru

Телефон \*

+79654198763

Количество

−

5

+

макс. 30 шт.

Итого:

**7 495 Р**

**Оформить заказ**

Рисунок 7 — Оформление заказа

Обычный пользователь может оформить заказ через модальное окно. В нем отображается название выбранной игрушки, цена за одну штуку, поля ФИО, email и телефона, а также выбор количества. Количество нельзя увеличить больше остатка товара на складе.

Перед отправкой формы клиентская часть проверяет, что ФИО заполнено, email имеет корректный вид, а телефон не пустой. После успешного оформления показывается сообщение об успешном создании заказа.

## 6.6 Раздел заказов

| <div>ГлавнаяКаталогЗаказыСтатистикаЛогии</div> |                                                                   |                                  |        |         |             |               |
|------------------------------------------------|-------------------------------------------------------------------|----------------------------------|--------|---------|-------------|---------------|
| №                                              | Покупатель                                                        | Игрушка                          | Кол-во | Сумма   | Статус      | Изменить      |
| #9                                             | Иордан Дмитрий Сергеевич<br>m2412976@edu.misis.ru<br>+79654198763 | LEGO City Полицейский участок    | 1 шт.  | 4 999 Р | Новый       | Новый ▾       |
| #2                                             | Иванов Иван<br>ivan@mail.ru<br>+79001112233                       | Монополия классическая           | 1 шт.  | 1 999 Р | Доставлен   | Доставлен ▾   |
| #3                                             | Петрова Анна<br>anna@mail.ru<br>+79004445566                      | Барби Модница                    | 2 шт.  | 2 998 Р | Отправлен   | Отправлен ▾   |
| #4                                             | Сидоров Пётр<br>petr@mail.ru<br>+79007778899                      | LEGO Technic Экскаватор          | 1 шт.  | 7 999 Р | Подтверждён | Подтверждён ▾ |
| #1                                             | Иванов Иван<br>ivan@mail.ru<br>+79001112233                       | LEGO City Полицейский участок    | 1 шт.  | 4 999 Р | Доставлен   | Доставлен ▾   |
| #6                                             | Козлова Мария<br>maria@mail.ru<br>+79009998877                    | Медведь плюшевый 60см            | 3 шт.  | 2 697 Р | Доставлен   | Доставлен ▾   |
| #7                                             | Козлова Мария<br>maria@mail.ru<br>+79009998877                    | Единорог мягкий                  | 2 шт.  | 1 398 Р | Отменён     | Отменён ▾     |
| #8                                             | Новиков Алексей<br>alex@mail.ru<br>+79003334455                   | Машинка Ferrari радиоуправляемая | 1 шт.  | 2 999 Р | Отправлен   | Отправлен ▾   |
| #5                                             | Сидоров Пётр<br>petr@mail.ru<br>+79007778899                      | Шахматы деревянные               | 1 шт.  | 799 Р   | Новый       | Новый ▾       |

Рисунок 8 — Раздел заказов

Раздел заказов доступен только администратору. В нем отображается таблица заказов. В таблице показаны номер заказа, данные покупателя, название игрушки, количество, сумма и текущий статус. Администратор может изменить статус заказа через выпадающий список.

### 6.7 Раздел статистики

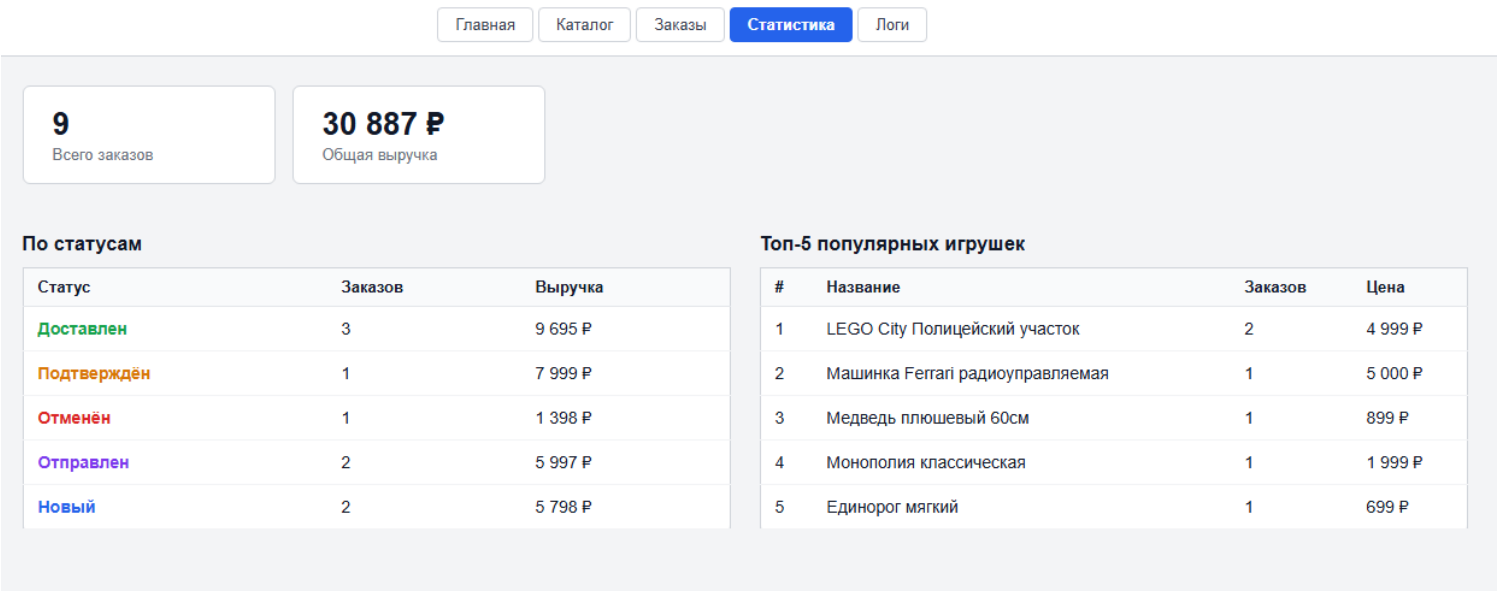


Рисунок 9 — Раздел статистики

Раздел статистики также доступен только администратору. В нем отображается общее количество заказов и общая выручка. Дополнительно есть таблица по статусам заказов и топ-5 популярных игрушек. Это помогает администратору быстро оценить работу магазина.

### 6.8 Раздел логов

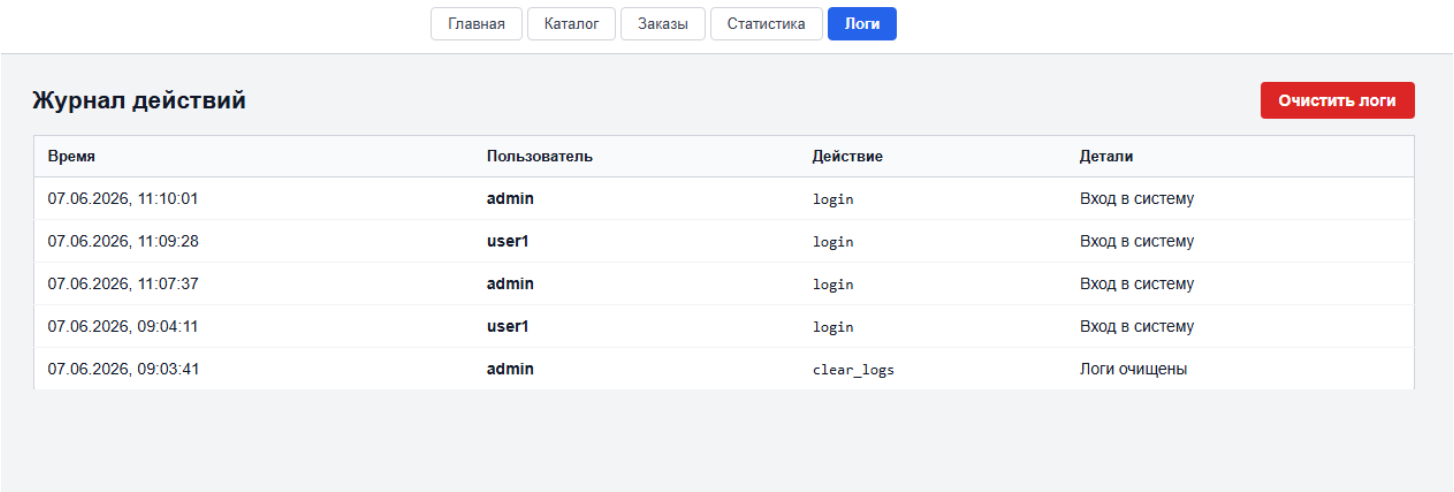


Рисунок 10 — Раздел логов

В разделе логов администратор видит журнал действий. В таблице отображаются время, пользователь, действие и детали. Также реализована кнопка очистки логов. Перед очисткой появляется окно подтверждения, чтобы пользователь случайно не удалил данные.

## 6.9 Роли в интерфейсе

Интерфейс меняется в зависимости от роли пользователя. Обычный пользователь видит только главную страницу и каталог. Администратор видит дополнительные вкладки: заказы, статистика и логи. Это делает интерфейс более понятным, потому что пользователь не видит функции, которые ему недоступны.

Таблица 17 — Доступность разделов по ролям

| Раздел     | Обычный пользователь | Администратор                             |
|------------|----------------------|-------------------------------------------|
| Главная    | Да                   | Да                                        |
| Каталог    | Просмотр и заказ     | Просмотр, добавление, изменение, удаление |
| Заказы     | Нет                  | Да                                        |
| Статистика | Нет                  | Да                                        |
| Логи       | Нет                  | Да                                        |

## 7. РАЗВЕРТЫВАНИЕ И ПРОВЕРКА РАБОТЫ СИСТЕМЫ

### 7.1 Контейнеризация

Для удобного запуска проекта используется Docker Compose. В docker-compose.yml описаны три сервиса: база данных PostgreSQL, backend на FastAPI и frontend на React. Такой подход позволяет запустить всю систему одной командой.

Таблица 18 — Сервисы Docker Compose

| Сервис   | Назначение              | Порт      |
|----------|-------------------------|-----------|
| db       | База данных PostgreSQL  | 5433:5432 |
| backend  | Серверная часть FastAPI | 8000:8000 |
| frontend | Клиентская часть React  | 3000:3000 |



Запуск проекта выполняется командой:

**`docker-compose up --build`**

После запуска становятся доступны следующие адреса:

- клиентская часть — `http://localhost:3000`;
- серверное API — `http://localhost:8000`;
- документация Swagger — `http://localhost:8000/docs`;
- PostgreSQL — `localhost:5433`.

## 7.2 Проверка авторизации

Для проверки авторизации можно использовать тестовые учетные записи. Администратор входит под логином `admin` и паролем `admin123`. Обычный пользователь входит под логином `user1` и паролем `user123`. После входа интерфейс показывает имя пользователя и его роль.

## 7.3 Проверка каталога

Для проверки каталога нужно открыть вкладку «Каталог». Система должна загрузить список игрушек из базы данных. В карточках должны отображаться цена, производитель, категория и остаток на складе. Поиск должен находить товары по названию или описанию.

## 7.4 Проверка оформления заказа

Проверка оформления заказа выполняется от имени обычного пользователя. Нужно выбрать товар в наличии, нажать «Заказать», заполнить ФИО, email, телефон и выбрать количество. После отправки формы должен появиться текст об успешном оформлении заказа. При этом остаток товара в каталоге должен уменьшиться.

## 7.5 Проверка панели администратора

Проверка панели администратора выполняется под учетной записью `admin`. Нужно открыть каталог и попробовать добавить новый товар, изменить существующий товар и удалить ненужный товар. Затем нужно

перейти в раздел заказов и изменить статус заказа. После этого в разделе логов должны появиться соответствующие записи.

## 7.6 Примеры тестовых сценариев

Таблица 19 — Тестовые сценарии

| № | Сценарий                                | Ожидаемый результат                          |
|---|-----------------------------------------|----------------------------------------------|
| 1 | Вход с правильным логином и паролем     | Пользователь попадает на главную страницу    |
| 2 | Вход с неправильным паролем             | Отображается ошибка авторизации              |
| 3 | Поиск товара по слову LEGO              | Показываются подходящие игрушки              |
| 4 | Оформление заказа с корректными данными | Создается заказ и уменьшается остаток товара |
| 5 | Оформление заказа больше остатка        | Сервер возвращает ошибку о недостатке товара |
| 6 | Добавление товара администратором       | Новый товар появляется в каталоге            |
| 7 | Изменение статуса заказа                | Статус в таблице заказов обновляется         |
| 8 | Просмотр логов                          | Администратор видит последние действия       |

## 8. ЗАКЛЮЧЕНИЕ

В ходе выполнения курсовой работы была разработана информационная система управления магазином детских игрушек «Toy Shop». Система позволяет автоматизировать основные процессы магазина: просмотр каталога, поиск товаров, оформление заказов, управление товарами, изменение статусов заказов, просмотр статистики и ведение журнала действий.

В рамках проекта была создана база данных PostgreSQL. В ней реализованы таблицы для пользователей, категорий, производителей, покупателей, игрушек, заказов и логов. Также были добавлены представления, функции и процедуры, которые помогают получать расширенную информацию и выполнять операции с данными.

Серверная часть была разработана на FastAPI. В ней реализована авторизация по JWT-токенам, проверка ролей, REST API для работы с товарами, заказами, статистикой и логами. Клиентская часть была разработана на React и предоставляет простой интерфейс для обычного пользователя и администратора.

Для удобного запуска проект был контейнеризирован с использованием Docker Compose. Это позволяет поднять базу данных, сервер и клиентскую часть одной командой. В системе также предусмотрены демонстрационные данные, поэтому приложение можно проверить сразу после запуска.

Таким образом, поставленная цель курсовой работы была достигнута. Разработанная система может быть использована как учебный пример клиент-серверного приложения, а также как основа для дальнейшего развития интернет-магазина игрушек.

В дальнейшем проект можно улучшить. Например, можно добавить корзину, несколько товаров в одном заказе, регистрацию новых пользователей, загрузку изображений игрушек, фильтрацию по цене и категории, а также более подробную аналитику продаж.

## 9. СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Официальная документация FastAPI. — URL: <https://fastapi.tiangolo.com>
2. Официальная документация SQLAlchemy. — URL: <https://docs.sqlalchemy.org>
3. Документация СУБД PostgreSQL 16. — URL: <https://www.postgresql.org/docs/16>
4. Официальная документация React. — URL: <https://react.dev>
5. Документация Docker Compose. — URL: <https://docs.docker.com/compose>
6. Документация библиотеки python-jose. — URL: <https://python-jose.readthedocs.io>
7. Зиборов В.В. PostgreSQL: примеры и задачи. — СПб.: БХВ-Петербург, 2022. — 352 с.
8. Любицкий В.Б. Разработка веб-приложений на FastAPI. — М.: ДМК Пресс, 2024. — 280 с.
9. Мартинес Х. React в действии. — СПб.: Питер, 2023. — 368 с.
10. Ссылка на GitHub с курсовой работой — <https://github.com/Dimas1973/toy-shop>

## ПРИЛОЖЕНИЕ А. ОСНОВНЫЕ API-МЕТОДЫ

В приложении реализован набор REST API-методов. Они используются клиентской частью для обмена данными с сервером. Ниже приведен общий список основных методов.

Таблица А.1 — Основные API-методы

| Раздел      | Метод  | URL                           | Описание                        |
|-------------|--------|-------------------------------|---------------------------------|
| Авторизация | POST   | /api/auth/login               | Вход пользователя               |
| Авторизация | GET    | /api/auth/me                  | Получение текущего пользователя |
| Игрушки     | GET    | /api/toys                     | Список игрушек                  |
| Игрушки     | GET    | /api/toys/search/{query_text} | Поиск игрушек                   |
| Игрушки     | GET    | /api/toys/category/{cat_id}   | Игрушки по категории            |
| Игрушки     | GET    | /api/toys/{toy_id}            | Одна игрушка                    |
| Игрушки     | POST   | /api/toys                     | Создание игрушки                |
| Игрушки     | PUT    | /api/toys/{toy_id}            | Изменение игрушки               |
| Игрушки     | DELETE | /api/toys/{toy_id}            | Удаление игрушки                |
| Покупатели  | GET    | /api/customers                | Список покупателей              |
| Покупатели  | POST   | /api/customers                | Создание покупателя             |
| Заказы      | GET    | /api/orders                   | Список заказов                  |
| Заказы      | POST   | /api/orders                   | Создание заказа                 |
| Заказы      | POST   | /api/orders/{order_id}/status | Изменение статуса               |
| Справочники | GET    | /api/categories               | Категории                       |
| Справочники | GET    | /api/manufacturers            | Производители                   |
| Статистика  | GET    | /api/stats/orders             | Статистика заказов              |
| Статистика  | GET    | /api/stats/popular            | Популярные товары               |
| Логи        | GET    | /api/logs                     | Журнал действий                 |
| Логи        | DELETE | /api/logs                     | Очистка журнала                 |

## ПРИЛОЖЕНИЕ Б. КРАТКОЕ ОПИСАНИЕ ФАЙЛОВ ПРОЕКТА

Проект состоит из нескольких основных файлов. Каждый файл отвечает за свою часть приложения.

*Таблица Б.1 — Основные файлы проекта*

| Файл               | Назначение                                                                                         |
|--------------------|----------------------------------------------------------------------------------------------------|
| main.py            | Основной файл серверной части FastAPI. Содержит маршруты API, авторизацию и работу с базой данных. |
| requirements.txt   | Список Python-зависимостей для backend.                                                            |
| init.sql           | Скрипт создания таблиц, представлений, функций, процедур и тестовых данных.                        |
| App.js             | Основной файл клиентской части React. Содержит интерфейс приложения и запросы к API.               |
| index.js           | Точка входа React-приложения.                                                                      |
| Dockerfile         | Описание сборки контейнера backend.                                                                |
| docker-compose.yml | Описание запуска базы данных, backend и frontend.                                                  |

Такое разделение делает проект понятным. Серверная часть находится отдельно от клиентской, а база данных описывается отдельным SQL-файлом. Благодаря Docker Compose систему можно быстро запустить на другом компьютере.